aliasing, text color, background color).

7. We create another object—an image we loaded.
8. The *pygame.event.get()* call is used to get all the events that are occurring. When it encounters QUIT (when the user closes the window), it will exit the loop, and close the display.
9. Blitting is like copying one surface to another surface. We copy the image surface object to the display surface object at the *(x, y)* co-ordinates. In the same way, we blit the font object to the display surface.
10. Blitting always takes place in a buffer; we will not be able to see its results. To display the result, we use this line of code, which updates the display from the buffer.

Figure 1: 'Hello World'

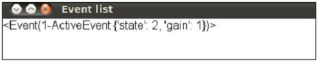

Figure 2: Event information

Events

Knowledge of event handling is very important if you are creating a game. Developers must know how their games ought to respond when a user clicks the mouse button, for instance. The number of events that can occur within the game are infinite. We will cover the major events, such as *MouseButtonDown, MouseButtonUp*, etc, with another sample script, *event.py*:

```
import pygame
from pygame.locals import *
from sys import exit
pygame.init()
a=pygame.display.set_mode((650, 400),RESIZABLE, 32)
pygame.display.set_caption("Event list ")
font=pygame.font.SysFont("arial",16)
back=pygame.Surface(a.get_size())
back=back.convert()
back.fill((255, 255, 255))
while True:
 for i in pygame.event.get():
  if i.type==QUIT:
   exit()
  else:
   text=font.render("%s"%i, 1,(10, 10, 10),(255, 255, 255))
  a.blit(back,(0, 0))
  a.blit(text,(0, 0))
  pygame.display.update()
```

The output is as shown in Figure 2.

Events are stored in the form of a queue. *Pygame.event.get()* gives us the list of occurring events within the queue. Running the above script will display all the events that occurred within the display, with all the information about the event. Let's go into the details for some of the important events:

- *MouseMotion:* It is a dictionary with the key elements *buttons*, *pos*, and *rel*. The key element *button* corresponds to a value like (0, 0, 0), in which the 0th element represents the mouse button on the left extreme, the 1st element the middle button, and the 3rd the right button. If clicked, the button value is 1; if not clicked, 0. The pos element represents the current coordinate of the mouse, and *rel* shows the difference in mouse position, with respect to the last event co-ordinates. You can move the mouse in the space with the mouse button held down, too.
- *MouseButtonDown:* The key elements are *button* and *pos*, with the same explanations as above.
- *MouseButtonUp:* This is similar to *MouseButtonDown*; the only difference is that *button* value is 0 instead of 1.
- *KeyDown:* The key elements of the dictionary are *scancode*, *key*, *unicode* and *mod*. When a key is pressed, this event is emitted. The *scancode* value is the platform-specific value of the *key* (key is the value of the key) and you can find out the values of all the keys in *pygame.locals*. mod indicates the combination mode (something like *Ctrl+x* will give a different mod value). *unicode* represents the Unicode value of the key that was pressed.
- *KeyUp:* Similar to *KeyDown*, but it doesn't contain the unicode element.

These events are used in almost every game. There are other events too, such as *videoresize*, etc. You can learn more about them on the Web. Pygame is of great use if you start your work from scratch—it's worth trying out this API. I am still reading about the module in depth, so I will try to provide readers with more knowledge about it in the future. Suggestions and queries are always welcome. END

By: Ankur Aggarwal

The author loves to explore open source technologies, and enjoys programming. He has learned C, C++, core Java, and Python. He loves rock/metal music, and has been trying his hand at the guitar. He has also started a blog recently (www.flossstuff.wordpress.com) and can be contacted via coolankur2006@gmail.com, or on Twitter as @ankurtwi.